Group 13: Basketball Detection for Televised Streams Using Machine Learning

Sarah MacGillivray (24787251)
Computer Engineering
UBC, Vancouver, BC
smacg04@gmail.com

Khai Phan (64849185)
Computer Engineering
UBC, Vancouver, BC
khaidoanhtkg@gmail.com

Glen Zhu (77148443)
Computer Engineering
UBC, Vancouver, BC
glen.zhu@outlook.com

Abstract

In the world of sports television, multiple camera angles, distracting crowds, and constant movement all vie for the viewer's attention. Basketball, being a fast-paced game, demands a reliable solution to identify the location of the ball on the screen. This can be the foundation of automatic scoring technologies or graphical enhancement technologies for visually impaired viewers. We set our goal to develop a machine learning model that can locate a basketball in any given frame with high emphasis on speed and accuracy. We utilize a dataset of 1,997 training images, 130 validation images and 472 testing images, containing images from various environments, lighting conditions and camera angles. Our baseline is SSD300 with a VGG16 backbone, which achieved 87.04% mAP@0.5 and 6.56ms/frame. To explore the best model(s) for our project, we conduct experiments on the following options: Faster-RCNN ResNet50, YOLOv8, and a weighted-box-fusion ensemble of combinations of options. Ultimately, YOLOv8 is our final model, achieving a mAP@0.5 score of 92.08% with an average latency of 6.59ms/frame - fast enough for inferencing on modern video frequencies.

1 Introduction

1.1 Motivation

Computer vision in sports analytics has become an increasingly active area of research, driven by the demand for a more fluid and automated broadcast experience. Recent advancements in computer vision have significantly impacted sports analytics by automating the collection, analysis, and interpretation of data from video footage, which traditionally relied heavily on manual effort. Video analysis of sports has yielded a wide range of applications, including extraction of ball trajectories, highlight detection, gameplay analysis, and referee decision support ([Fang et al., 2025]). Furthermore, people with low vision requires supportive technologies to enhance their perception and overall experience ([Lee et al., 2024]). Knowing where the ball is at any given moment is the foundational problem of these technologies and, therefore, is our motivation for this project.

In this paper, basketball will be the main focus due to its fast-paced nature and reasonably close-range camera angles, compared to other popular sports such as football or baseball.

1.2 Problem Definition

The problem we would like to solve is to develop a machine learning model that can detect the location of the basketball with a strong emphasis on accuracy (mAP@0.5 in the case of object detection) and speed. We do not concern other factors involved in the game, such as players, referees, rim, basket, etc.

Preprint.

The solution takes RGB images (3 channels) as input. The size of these images is of little importance since the preprocessing step will scale them to the appropriate size for each option. The output of the solution should be the bounding box enclosing pixels making up the basketball for each frame, the confidence, and optionally the class for multiclass detection. The specific output shape is dependent on the various model architectures we explore. However, all models produce a consistent final output after post-processing:

$$\{(x_1, y_1, x_2, y_2, s, c)\}_{i=1}^{N}$$

where x_1, y_1, x_2, y_2 are bounding box coordinates, confidence score s , and predicted class label c. N is the number of samples.

The metrics mAP@0.5 quantifies the localization accuracy of object detection compared to the labelled ground-truth set by counting a detection with confidence at least 50% as a true positive. This is the popular standard benchmark for a variety of object detection problems ([Mallick, 2022]).

Regarding speed, we set the benchmark to be modern TV broadcast frequencies, which are around 25-30 FPS, according to [Hullfish, 2025]. We expect the solution to produce at least the same frame rate or higher.

Recall, precision and F1-score are also some of the metrics that we are interested in, with a lower emphasis. These help us answer the questions of how the model performs in terms of not missing predictions, not hallucinating predictions, and a harmonic balance of both.

1.3 Contributions

After establishing the dataset for this task, we trained each model on 1,997 images, validating on 130 images and testing on 472 images. We then compare 4 different models: an SSD300 with a VGG16 Backbone, a Faster-RCNN with a ResNet-50 Backbone, YOLOv8, and an ensemble of the trained SSD300 and YOLOv8.

All models achieve above 86% in mAP@0.5, with our final solution, YOLOv8 (standalone), achieving **92.08% mAP@0.5** and **6.56 ms/frame or 152.4 FPS**.

2 Method

2.1 Algorithm Description

Development space for each model, done in notebooks, can be found in a collective github repository, found at the following link: Basketball Detection Repositiitory. The final trained models are in the $./models$ folder.

2.1.1 SSD300 with a VGG16 Backbone

The SSD (Single Shot MultiBox Detector) [Liu et al., 2016] is a single-stage object detector that predicts bounding boxes and class scores directly from multi-scale feature maps in a single forward pass, making it well-suited for speed-critical use cases. We used a VGG16 backbone [blurredmachine, 2020] pretrained on COCO as the feature extractor. VGG16, developed by the Visual Geometry Group at Oxford, is a deep convolutional network consisting of 13 convolutional layers using 3×3 filters and 3 fully connected layers, with max-pooling between convolutional blocks. The pretrained weights provide strong low-level feature extraction out of the box, which we then fine-tuned for our basketball detection.

We replaced the model's classification head with a new `SSDClassificationHead` configured for $K = 4$ classes (background, basketball, rim, and sports ball), with feature map input channels of $[512, 1024, 512, 256, 256, 256]$ across the six prediction scales.

Loss function. SSD is trained using a MultiBox detector that minimizes a weighted combination of a localization loss and a confidence loss [Liu et al., 2016]:

$$\mathcal{L}(\mathbf{x}, \mathbf{c}, \mathbf{l}, \mathbf{g}) = \frac{1}{N}\big(\mathcal{L}_{conf}(\mathbf{x}, \mathbf{c}) + \alpha\, \mathcal{L}_{loc}(\mathbf{x}, \mathbf{l}, \mathbf{g})\big) \tag{1}$$

where N is the number of matched default boxes and $\alpha = 1$ balances the two terms. The localization loss is a Smooth-ℓ_1 regression between the predicted box offsets $\mathbf{l}$ and the ground-truth box $\mathbf{g}$, and the confidence loss is a softmax cross-entropy over all class predictions [Liu et al., 2016].

Preprocessing. All images were resized to 300×300 pixels and normalized using ImageNet channel statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$). Bounding-box coordinates were rescaled proportionally from the original image resolution into 300×300 pixel space, and any bad boxes (where $x_{\max} \leq x_{\min}$ or $y_{\max} \leq y_{\min}$) were skipped during loading.

Optimizer and schedule. After iterating over learning rates $\{1 \times 10^{-3},\ 7.5 \times 10^{-4},\ 5 \times 10^{-4},\ 2.5 \times 10^{-4}\}$, epoch budgets of 15, 30, 50, 70, and 100, and batch sizes 8, 16, 32, and 64, the best configuration used SGD with a learning rate of 5×10^{-4}, momentum of 0.9, and weight decay of 5×10^{-4}, trained for 100 epochs with a batch size of 16. Higher learning rates such as 1×10^{-3} produced unstable training and frequent NaN losses, while lower values like 2.5×10^{-4} converged too slowly within the epoch budget. The 100-epoch run at 5×10^{-4} consistently achieved the best validation mAP@50 and was selected as the final configuration. To prevent early divergence, a linear warm-up was applied over the first 3 epochs, scaling the learning rate from 10% to 100% of its base value. After warm-up, cosine annealing [PyTorch Contributors, 2026] decayed the learning rate smoothly to zero:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{\pi\, t}{T_{\max}}\right) \tag{2}$$

Mixed-precision training (`torch.amp`) was used to reduce GPU memory usage and speed up training. Gradient clipping ($\|\nabla\|_2 \leq 1.0$) was also applied, as early runs without it produced exploding gradients that caused NaN losses and destabilized the training loop entirely.

2.1.2 Faster-RCNN with a ResNet-50 backbone

The Faster R-CNN (Region-Based Convolutional Neural Network) [Ren et al., 2015] is a two-stage object detector that first generates region proposals and then classifies and refines them. Unlike single-stage detectors such as SSD, Faster R-CNN prioritizes detection accuracy by separating proposal generation and classification, increasing the computational overhead. We used a ResNet-50 backbone pretrained on COCO as the feature extractor. ResNet-50 is a deep convolutional network consisting of 50 layers with residual (skip) connections that mitigate vanishing gradients and enable stable training of deeper architectures. These pretrained weights provide strong hierarchical feature representations, which were fine-tuned for our basketball detection task.

The model consists of two main components: a Region Proposal Network (RPN) and a detection head. The RPN operates on backbone feature maps to generate candidate object regions, predicting both objectness scores and bounding box offsets. These proposals are processed and passed through fully connected layers for classification and bounding box regression. The classification head was configured for `K = num_classes = 2` (as defined in training), including background.

Loss function. Faster R-CNN is trained using a multi-task loss that combines classification and localization terms from both the RPN and the detection head:

$$L = L_{RPN} + L_{det}$$

Each component consists of a classification loss (cross-entropy for objectness or class prediction) and a localization loss (Smooth-L1 for bounding box regression). During training, the model returns a dictionary of these losses, which are summed to compute the final optimization objective.

Preprocessing. Images were transformed using a custom augmentation pipeline, with separate configurations for training and validation. Faster R-CNN operates on variable-sized images, with internal resizing handled by the model. Images were normalized implicitly according to the pretrained backbone requirements. Bounding-box coordinates were preserved in the original image scale, and invalid boxes were filtered during dataset loading.

Optimizer and schedule. Training used the AdamW optimizer with weight decay of 1×10^{-4}. Different learning rates were applied to different parts of the model: 1×10^{-4} for backbone (fine-tuned layers), and 5×10^{-4} for detection head and RPN.

To stabilize training and preserve pretrained features, the early backbone layers were frozen, specifically: `conv1, bn1, layer1`, and `layer2`, while deeper layers (from `layer3` onward) were fine-tuned.

The model was trained for `num_epoch=20` epochs with a training batch size of 4 and a validation batch size of 2.

A cosine annealing learning rate scheduler was applied over the full training duration:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{\pi t}{\texttt{num_epoch}}\right)\right) \tag{3}$$

Training stability. Standard full-precision training was used (no mixed precision). Gradient clipping was not applied, as the AdamW optimizer provided stable convergence without exploding gradients in practice. The model was trained end-to-end with loss monitoring, and the best checkpoint was selected based on lowest training loss, rather than validation mAP.

2.1.3 YOLOv8

YOLOv8 by Ultralytics is the industry standard object detection model, known for its high efficiency and accuracy. This model processes an image through a convolutional neural network (CNN) in a single pass to predict bounding boxes, objectness score, and class probabilities. There are three main components for this model: the backbone (CSPDarknet53) for feature extraction, the neck (PANet) that aggregates feature maps across different scales, and the head: an anchor free split-head design that directly predicts the center of objects and their size.

Loss function. YOLOv8 utilizes a composite of loss functions designed optimize localization, classification, and boundary precision. The first loss function which targets localization is the complete intersection over union (CIoU) loss function [Min-soo et al., 2023]:

$$\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v \tag{4}$$

where b and b^{gt} denote the central points of B (predicted box) and B^{gt} (ground truth), $\rho(\cdot)$ is the Euclidean distance, c is the diagonal length of the smallest enclosing box covering the two boxes, α is a positive trade-off parameter, and v measures the consistency of aspect ratio.

The parameter α is a positive trade-off term where overlaps are given higher priority over non-overlap cases, and v provides information about the consistency of the aspect ratio.

The next loss function that focuses on optimizing classification is the binary cross-entropy function. For each class i, the model compares the ground truth label y_i (1 if the class is present, 0 otherwise) with the predicted probability $\hat{p}_i$, allowing it to assign multiple labels to a single object when necessary:

$$\mathcal{L}_{cls} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\sigma(x_i)) + (1 - y_i)\log(1 - \sigma(x_i))] \tag{5}$$

Lastly, the loss function that targets boundary precision is the distributed focal loss function. Instead of predicting a single coordinate, YOLOv8 predicts a probability distribution around the ground truth to handle "fuzzy" boundaries. If the ground truth value is y, and it falls between two discrete "bins" y_i and y_{i+1} ($y_i \leq y \leq y_{i+1}$), DFL forces the model to shift more probability to these two nearest neighbors:

$$\mathcal{L}_{DFL}(P_i, P_{i+1}) = -((y_{i+1} - y)\log(P_i) + (y - y_i)\log(P_{i+1})) \tag{6}$$

Preprocessing. Dataset images are processed by YOLOv8 through a pipeline of image-resizing, normalization, and colour format conversion. Images are typically resize to 640×640, normalization for pixel values from $[0, 255]$ to $[0, 1]$, and converting colour formats from BGR to RGB when necessary.

Optimizer and schedule. YOLOv8 is able to deploy a wide range of optimizers such as Adam, or AdamW, but for our case we used a stochastic gradient descent (SGD) optimizer during our training phase. We started off with a $lr_0 = 0.01$ (starting learning rate) and had a cosine decay scheduler:

$$\mathrm{lr}(t) = \mathrm{lr}_0 \cdot \left(\frac{1 + \cos\left(\frac{\pi t}{T}\right)}{2} \cdot (1 - \mathrm{lrf}) + \mathrm{lrf} \right) \tag{7}$$

where t is the current epoch and T is the total number of epochs and lrf is the final learning rate factor.

2.1.4 Weighted Box Fusion Ensemble

To further improve detection performance, we implemented a Weighted Box Fusion (WBF) ensemble combining the YOLOv8 and SSD300 models described above. While we also experimented with additional combinations, such as Faster R-CNN with YOLOv8, the YOLOv8 + SSD300 ensemble provided the best trade-off between accuracy and computational efficiency. Other configurations are included in our codebase for completeness.

To enable ensembling, outputs from both models were reformatted into a unified representation, ensuring consistent bounding box coordinates, confidence scores, and class labels. A short warm-up phase (300 iterations) was applied using a dummy tensor to stabilize inference and ensure consistent runtime behavior across models.

Unlike traditional ensembling methods such as Non-Maximum Suppression (NMS), which discard overlapping detections, WBF aggregates predictions by clustering bounding boxes with high Intersection over Union (IoU). Boxes within the same cluster are merged into a single prediction using a weighted average based on both confidence scores and model-specific weights. This results in more precise localization and better utilization of complementary model predictions.

Formally, the fused bounding box coordinates are computed as [Jadhav et al., 2025]:

$$coord_i^{final} = \frac{\sum_{j=1}^{n} s_j^i \cdot w_j \cdot coord_j^i}{\sum_{j=1}^{n} s_j^i \cdot w_j} \tag{8}$$

where s_j^i is the confidence score, w_j is the model weight, and $coord_j^i$ represents the i-th coordinate of the j-th bounding box in the cluster.

The final confidence score is computed as [Jadhav et al., 2025]:

$$s_i^{final} = \frac{\max_{j=1}^{n}(s_j^i \cdot w_j)}{\max_{j=1}^{n}(w_j)} \tag{9}$$

This approach allows the ensemble to retain high-confidence detections while improving bounding box alignment.

2.2 Overall Framework and Diagram

2.2.1 SSD300 and Faster R-CNN

Figure 1 illustrates the shared training and inference pipeline for SSD300 and Faster R-CNN. During training, images are preprocessed per model: SSD300 resizes inputs to 300×300 and applies ImageNet channel-wise normalization ($x' = (x - \mu)/\sigma$), while Faster R-CNN applies random horizontal flipping and ensures RGB format. Both models are initialized with pretrained backbone weights, VGG16 for SSD300 (all layers trainable) and ResNet-50 for Faster R-CNN (early layers `conv1`, `bn1`, `layer1-2` frozen). Training runs for the configured number of epochs and the checkpoint with the best validation mAP@0.5 is saved. During inference, batches are loaded without augmentation, the best checkpoint generates predictions, and mAP@0.5, precision, recall, F1, and FPS are recorded.

Design Choices. SSD300 requires a fixed 300×300 input to align with its predefined multi-scale anchor layers, which helps ensure efficient batch processing and consistent feature map scaling. ImageNet normalization aligns the input distribution with the pretrained VGG16 backbone, improving

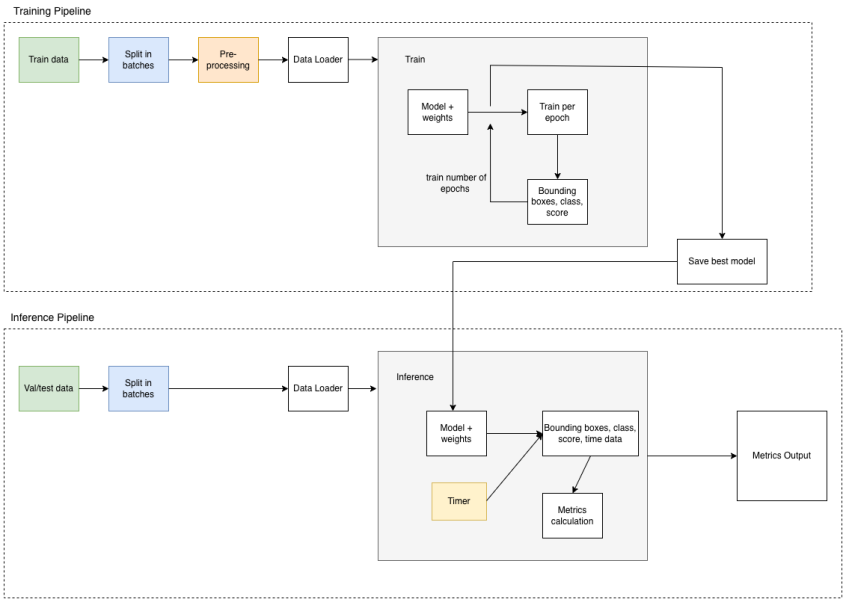

Figure 1: Training and inference pipeline for SSD300 and Faster R-CNN.

convergence and allowing effective transfer learning. This also allows the image to be centered and scales pixel intensities so that the model is less sensitive to illumination. Faster R-CNN accepts variable-sized inputs, preserving the original aspect ratio for more accurate region proposals. Normalization is handled internally, which means that images are converted to RGB which makes for a consistent colour space that helps against feature mismatch and lets colour filters be applied correctly. Horizontal flipping augments orientation diversity, which is beneficial in broadcast scenarios where the ball can appear anywhere on screen, so localization can occur without distortion-induced bias.

2.2.2 YOLOv8

Figure 2 shows the YOLOv8 pipeline. During training, images are resized to 640×640 via letterbox resizing, normalized to $[0, 1]$, and augmented with mosaic augmentation, random scaling, horizontal flipping, and color jitter. The model is initialized with COCO pretrained weights, all layers are trainable, and training runs for 50 epochs with the best mAP@0.5 checkpoint saved. During inference, images are resized and normalized without augmentation, and Non-Maximum Suppression (NMS) is applied with $conf = 0.3$ and $IoU = 0.5$ to remove redundant detections.

Design Choices. Letterbox resizing preserves the original aspect ratio while meeting the fixed input requirement, avoiding geometric distortion that could harm localization. Mean IoU is excluded from evaluation as it is not natively supported by YOLOv8's validation API; mAP@0.5 is the preferred metric as it captures both localization and classification accuracy across all classes.

3 Experiment

3.1 Experiment Setup

For thorough selection of a primary model, the experiment was developed to test and optimize three models, along with attempting an ensemble with the finalized SSD300 and YOLOv8 models. The three candidates were SSD300 with a VGG16 backbone, Faster-RCNN ResNet50, and YOLOv8. Each model was individually tested and tuned to perform as well as possible before any comparison was made.

The dataset split was held consistent across all models to allow for reliable comparison. The train/validation/test split was pre-established by the Roboflow dataset, which contained 1,997 training images, 130 validation images, and 472 testing images, which breaks down to a 77% train, 5%

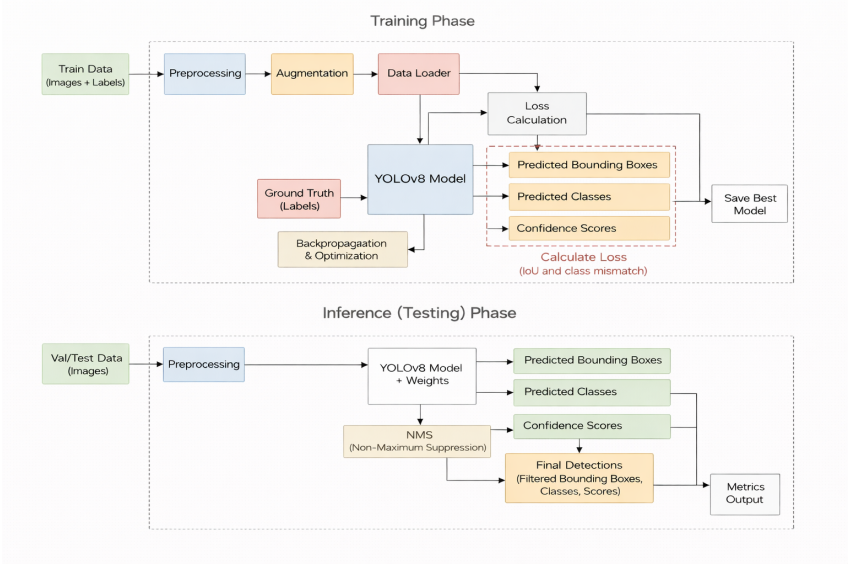

Figure 2: Training and inference pipeline for YOLOv8.

validation, and 18% test split[Eagle Eye, 2024]. The dataset captures basketballs across a wide variety of positions, lighting conditions, and camera angles, making it well-suited for simulating real-time television broadcast scenarios. All models were trained on a Kaggle T4 GPU environment. The primary metric used during training was mAP@50, with final model selection based on a combination of Precision, Recall, F1 score, mean IoU, and inference speed.

3.2 Model Comparison and Results

Table 1 summarizes the performance of all four models evaluated on the 472-image test set. Metrics include Precision, Recall, F1 Score, Mean IoU, and inference speed (FPS), which are the most relevant measures for our real-time basketball detection objective.

Table 1: Test set performance comparison across all models on 472 images. FPS is measured on a Kaggle T4 GPU. Higher is better for all metrics.

Model	**Precision**	**Recall**	**F1**	**Mean IoU**	**FPS**	**mAP@0.5**
SSD300 VGG16	0.8608	0.8109	0.8351	0.8386	57.09	0.8704
Faster-RCNN ResNet50	0.8635	0.7975	0.8292	0.8498	12.30	0.8677
YOLOv8	0.9715	0.9167	0.9433	—	158.3	0.9208
WBF Ensemble	0.9473	0.8408	0.8909	0.8654	57.09	0.9056

As seen in Table 1, SSD300 and Faster-RCNN perform similarly across most metrics. Faster-RCNN edges out SSD300 slightly in Precision (0.8635 vs 0.8608) and Mean IoU (0.8498 vs 0.8386), but SSD300 wins in Recall (0.8109 vs 0.7975) and F1 score (0.8351 vs 0.8292), while being nearly five times faster at 57.09 FPS compared to Faster-RCNN's 12.30 FPS, which goes below our 25–30 FPS broadcast benchmark.

YOLOv8 outperforms every other model by a significant margin across all available metrics, achieving a Precision of 0.9715, Recall of 0.9167, and F1 score of 0.9433, while also being the fastest at 158.3 FPS. This makes it both the most accurate and most deployable model for real-time basketball detection. The WBF ensemble placed second in F1 score at 0.8909 and achieved the lowest false positive count of any model (FP: 25), showing that combining models does help precision, but it still falls short of YOLOv8 on every metric.

3.3 Justification and Analysis

Overall, YOLOv8 was selected as our final model, as it outperformed every other candidate across all available metrics. This result is consistent with the architectural advantages of the YOLO family. Unlike Faster-RCNN, which uses a two-stage pipeline, first proposing candidate regions, then classifying them. YOLOv8 performs detection in a single forward pass, predicting bounding boxes and class scores simultaneously. This not only makes it significantly faster, but also allows the model to reason over the full image context at once, rather than evaluating isolated region proposals independently. SSD also performs single-stage detection, but differs from YOLO in that it applies predictions across fixed convolutional feature map regions at multiple scales, whereas YOLO uses a more holistic view of the entire image during prediction, which likely contributes to its stronger Recall and Precision in our experiments.

The ensemble model theoretically explores the combining strengths of SSD300 and YOLOv8. However, since its metrics are on-par or lower than those of YOLOv8 standalone (Table 1), it shows that there is some overlap in the shortcomings of YOLOv8 and SSD300. Moreover, SSD300 is slower than YOLOv8, which creates a latency bottleneck. This further justifies the usage of YOLOv8.

Given that our objective prioritizes real-time detection for live television broadcasts, both speed and precision are critical. A false positive, detecting a basketball that is not there, risks distracting the viewer or triggering incorrect downstream processing, while a false negative means the ball is missed entirely. YOLOv8's Precision of 0.9715 and Recall of 0.9167 give great balance of both, and at 158.3 FPS it exceeds the 25–30 FPS broadcast benchmark, making it the clear choice for deployment.

However, the YOLOv8 model also makes mistakes. Some cases: (1) When the camera is too far away or when there is motion blur (false negative); (2) When the players' hands are on the ball, creating unpredictable texture (false negative); and (3) When there is a ball on the player's team logo (false positive).

4 Conclusion

4.1 Summary

We evaluated four basketball detection models, SSD300 (VGG16), Faster R-CNN (ResNet-50), YOLOv8, and a WBF ensemble of SSD300 and YOLOv8, for real-time basketball detection in televised broadcasts. YOLOv8 emerged as the strongest model, achieving 92.08% mAP@0.5, a precision of 0.9715, recall of 0.9167, and an inference speed of 158.3 FPS, well exceeding the 25–30 FPS broadcast requirement. Its single-stage, full-image architecture provided both the accuracy and speed necessary for real-time deployment, outperforming all other candidates by a substantial margin.

4.2 Limitations and Future Work

The primary limitation of our approach is the relatively small and domain-specific dataset (1,997 training images), which may limit generalization to uncommon broadcast styles, lighting conditions, or camera angles not represented in training. Additionally, our evaluation was conducted on static frames rather than true video streams, so temporal aspects such as motion blur consecutive frames remain unaddressed. Future work could incorporate temporal models (e.g., convolutional LSTM or tracking algorithms such as DeepSORT) to improve detection continuity across frames, expand to multi-sport detection, and validate performance on live broadcast hardware.

5 Distribution of Work

Sarah MacGillivray: Worked on developing and fine-tuning the SSD model and wrote the report along side my teammates.
Khai Phan: Worked on developing and fine-tuning the Faster RCNN model and the SSD - YOLOv8 ensemble model and wrote the report.
Glen Zhu: Collected the dataset, worked on developing and fine-tuning YOLOv8 model, format code, and wrote the report.

References

blurredmachine. Vggnet-16 architecture: A complete guide. 2020. Kaggle notebook, accessed April 14, 2026.

Eagle Eye. Basketball dataset. https://universe.roboflow.com/eagle-eye/basketball-1zhpe/dataset/1, 2024.

C. Fang, C. Moreira, X. Perez-Sala, and H. Possegger. *Computer Vision for Sports Analytics*. Springer Nature, 2025.

S. Hullfish. What are progressive frames, interlaced frames and the field rate? *Wolfcrow*, 2025. Online; accessed: 2026.

Atharva Jadhav, Arush Karekar, Manas Divekar, and Shachi Natu. Ssd ensemble with weighted boxes fusion for weapon detection. https://arxiv.org/abs/2509.23697, 2025. arXiv preprint, accessed April 15, 2026.

J. Lee, Y. Li, D. Bunarto, E. Lee, O. H. Wang, A. Rodriguez, Y. Zhao, Y. Tian, and J. E. Froehlich. Towards ai-powered ar for enhancing sports playability for people with low vision: An exploration of arsports. In *IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*, 2024.

Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. SSD: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*, 2016.

S. Mallick. Mean average precision (map) in object detection. *LearnOpenCV*, 2022. Online; accessed: 2026.

Ko Min-soo, Ko Dae-hwa, JE LEE, and Deok Jong Moon. Loss / ciou loss - bof - deep learning bible. https://wikidocs.net/167839, 2023. Accessed April 15, 2026.

PyTorch Contributors. torch.optim.lr_scheduler.cosineannealinglr. https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.CosineAnnealingLR.html, 2026. PyTorch documentation, accessed April 14, 2026.

Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.